Documentation technique — Portfolio

Nov 2, 2026 · @Williams

Le portfolio est une application React statique hébergée sur Vercel, qui s'appuie sur Supabase pour le contenu administrable et sur EmailJS pour le contact.

Architecture



Architecture · navigateur et 4 services

Le site n'a pas de serveur applicatif : l'application s'exécute dans le navigateur, et la sécurité des données repose sur les règles RLS de Supabase. Routes : `/` pour le site public, `/admin` pour la connexion, `/admin/dashboard` pour l'administration.

Installation et déploiement

Prérequis : Node.js 22 ou plus récent et npm. Le dépôt est `github.com/Wil-01/portfolio-nextgen`.

1. Installer les dépendances : `npm install`.
2. Copier `.env.example` vers `.env`, puis renseigner les cinq variables ci-dessous.
3. Exécuter `supabase-schema.sql` dans l'éditeur SQL de Supabase.
4. Créer le compte administrateur dans Supabase Auth, puis désactiver les inscriptions publiques.
5. Lancer `npm run dev`.

Variable	Rôle
`VITE_SUPABASE_URL`	URL du projet Supabase
`VITE_SUPABASE_ANON_KEY`	Clé publique, protégée par les règles RLS
`VITE_EMAILJS_SERVICE_ID`	Service d'envoi EmailJS
`VITE_EMAILJS_TEMPLATE_ID`	Modèle du message de contact
`VITE_EMAILJS_PUBLIC_KEY`	Clé publique EmailJS

Ces variables sont intégrées au JavaScript public au moment du build. Aucun secret ne doit porter le préfixe `VITE_`.

Déploiement. Vercel construit le projet avec `npm run build` et publie `dist/`. `vercel.json` renvoie toutes les routes vers `index.html`, ce qui permet d'ouvrir `/admin` directement. Après un changement de variable, il faut redéployer.

Commande	Rôle
`npm test`	Tests unitaires et d'intégration
`npm run test:coverage`	Tests avec couverture
`npm run lint`	Analyse ESLint
`npm run monitor`	Contrôle de la production

Données et sécurité

La lecture est publique et l'écriture réservée aux utilisateurs authentifiés, par des règles RLS sur chaque table.

Table	Contenu	Colonnes clés
`projects`	Fiches projet	`category`, `title`, `stack[]`, `axes[]`, `context`, `what_i_did[]`, `learned`, `order_index`
`skill_categories`	Familles de compétences	`label`, `skills[]`, `badge_cls`, `head_cls`, `order_index`

Table	Contenu	Colonnes clés
`experiences`	Expériences et formations	`type`, `period`, `is_current`, `title`, `organization`, `points[]`, `tags[]`, `order_index`

Les tables sont indépendantes : le schéma ne déclare aucune clé étrangère. Le bucket public `media` stocke les images et les vidéos envoyées depuis l'administration.

Authentification. `AuthContext` gère la session Supabase et `ProtectedRoute` redirige les visiteurs anonymes vers `/admin`. Ce contrôle côté client ne sert qu'à l'affichage : la vraie protection des données repose sur les règles RLS.

Point de vigilance. Les règles acceptent tout utilisateur authentifié, pas seulement l'administrateur. Les inscriptions publiques doivent donc rester désactivées.

Structure du code et points d'attention

Dossier	Contenu
`src/pages/`	Sections du one page (`Home`, `About`, `Projects`, `Resume`, `Contact`)
`src/pages/admin/`	Connexion, tableau de bord et onglets Projets, Compétences, Parcours
`src/components/`	Navigation, cartes et fenêtre de projet, thème, champs d'administration
`src/hooks/usePortfolioData.ts`	Lecture Supabase et conversion vers le modèle du front
`src/data/projectData.ts`	Projets statiques de secours
`src/test/`	Configuration des tests et simulation de Supabase
`scripts/smoke-check.mjs`	Contrôle de disponibilité

Contenu de secours. Les projets, les compétences et le parcours existent en version statique dans le code. Si Supabase échoue ou si une table est vide, le site public affiche cette version sans message d'erreur. Une panne de la base reste donc invisible pour le visiteur, d'où l'intérêt du contrôle automatique.

Découpage du JavaScript. Les pages d'administration sont chargées à la demande et les bibliothèques volumineuses sont séparées dans `vite.config.ts`.

Anomalies ouvertes au 2 novembre 2026 :

- Le domaine Supabase configuré en production n'existe plus, ce qui rend l'administration inutilisable.
- Le formulaire de projet perd le focus pendant la saisie, car `Form` est déclaré dans `ProjectsTab`.
- `dbToProject` utilise `order_index` comme identifiant, ce qui crée des doublons.
- `ProjectsTab.save` ignore les erreurs d'écriture.
- `npm run lint` signale 6 erreurs et 3 avertissements dans le code existant.